In [ ]:

```
                                                    Practical No : 02

Name : Dhanke Anuj Mahadev
Roll No : 20
Batch : A1
Problem Statement : Classify the email using the binary classification method. Email Spa
detection has two states:
a)Normal State – Not Spam,
b) Abnormal State – Spam.
Use K-Nearest Neighbors and Support VectorMachine for classification. Analyze
their performance.
Dataset link: The emails.csv dataset on the Kaggle
https://www.kaggle.com/datasets/balaka18/email-spam-classification-dataset-csv
```

In [3]:

```
import pandas as pd
import numpy as np
import seaborn as sns
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisp
```

In [4]:

```
data = pd.read_csv("emails.csv")
```

In [5]:

```
data
```

Out[5]:

	Email No.	the	to	ect	and	for	of	a	you	hou	...	connevey	jay	valued	lay	infrastructure
0	Email 1	0	0	1	0	0	0	2	0	0	...	0	0	0	0	0
1	Email 2	8	13	24	6	6	2	102	1	27	...	0	0	0	0	0
2	Email 3	0	0	1	0	0	0	8	0	0	...	0	0	0	0	0
3	Email 4	0	5	22	0	5	1	51	2	10	...	0	0	0	0	0
4	Email 5	7	6	17	1	5	2	57	0	9	...	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
5167	Email 5168	2	2	2	3	0	0	32	0	0	...	0	0	0	0	0
5168	Email 5169	35	27	11	2	6	5	151	4	3	...	0	0	0	0	0
5169	Email 5170	0	0	1	1	0	0	11	0	0	...	0	0	0	0	0
5170	Email 5171	2	7	1	0	2	1	28	2	0	...	0	0	0	0	0

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

	Email No.	the	to	ect	and	for	of	a	you	hou	...	connevey	jay	valued	lay	infrastructure
5171	Email 5172	22	24	5	1	6	5	148	8	2	...	0	0	0	0	0

5172 rows × 3002 columns

In [6]:

```
data.shape
```

Out[6]:

```
(5172, 3002)
```

In [7]:

```
data = data.drop('Email No.', axis=1)
```

In [8]:

```
data.head
```

Out[8]:

```
<bound method NDFrame.head of       the  to  ect  and  for  of    a  you  hou  in  ...
connevey  jay  \
0         0   0    1    0    0   0    2    0    0   0  ...         0    0
1         8  13   24    6    6   2  102    1   27  18  ...         0    0
2         0   0    1    0    0   0    8    0    0   4  ...         0    0
3         0   5   22    0    5   1   51    2   10   1  ...         0    0
4         7   6   17    1    5   2   57    0    9   3  ...         0    0
...     ...  ..  ...  ...  ...  ..  ...  ...  ...  ..  ...       ...  ...
5167      2   2    2    3    0   0   32    0    0   5  ...         0    0
5168     35  27   11    2    6   5  151    4    3  23  ...         0    0
5169      0   0    1    1    0   0   11    0    0   1  ...         0    0
5170      2   7    1    0    2   1   28    2    0   8  ...         0    0
5171     22  24    5    1    6   5  148    8    2  23  ...         0    0

      valued  lay  infrastructure  military  allowing  ff  dry  Prediction
0          0    0               0         0         0   0    0           0
1          0    0               0         0         0   1    0           0
2          0    0               0         0         0   0    0           0
3          0    0               0         0         0   0    0           0
4          0    0               0         0         0   1    0           0
...      ...  ...             ...       ...       ...  ..  ...         ...
5167       0    0               0         0         0   0    0           0
5168       0    0               0         0         0   1    0           0
5169       0    0               0         0         0   0    0           1
5170       0    0               0         0         0   1    0           1
5171       0    0               0         0         0   0    0           0

[5172 rows x 3001 columns]>
```

In [9]:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5172 entries, 0 to 5171
Columns: 3001 entries, the to Prediction
dtypes: int64(3001)
memory usage: 118.4 MB
```

In [10]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
data.describe()
```

Out[10]:

	the	to	ect	and	for	of	a	
count	5172.000000	5172.000000	5172.000000	5172.000000	5172.000000	5172.000000	5172.000000	51
mean	6.640565	6.188128	5.143852	3.075599	3.124710	2.627030	55.517401	
std	11.745009	9.534576	14.101142	6.045970	4.680522	6.229845	87.574172	
min	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	1.000000	1.000000	0.000000	1.000000	0.000000	12.000000	
50%	3.000000	3.000000	1.000000	1.000000	2.000000	1.000000	28.000000	
75%	8.000000	7.000000	4.000000	3.000000	4.000000	2.000000	62.250000	
max	210.000000	132.000000	344.000000	89.000000	47.000000	77.000000	1898.000000	

8 rows × 3001 columns

In [11]:

```
data['Prediction'].value_counts()
```

Out[11]:

```
Prediction
0    3672
1    1500
Name: count, dtype: int64
```

In [12]:

```
X = data.drop('Prediction', axis=1)
y = data['Prediction']
```

In [13]:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)
```

In [14]:

```
neigh = KNeighborsClassifier(n_neighbors=2)
neigh.fit(X_train, y_train)
```

Out[14]:

▾ KNeighborsClassifier

▶ Parameters

In [15]:

```
y_pred = neigh.predict(X_test)
```

In [16]:

```
neigh.score(X_train, y_train)
neigh.score(X_test, y_test)
```

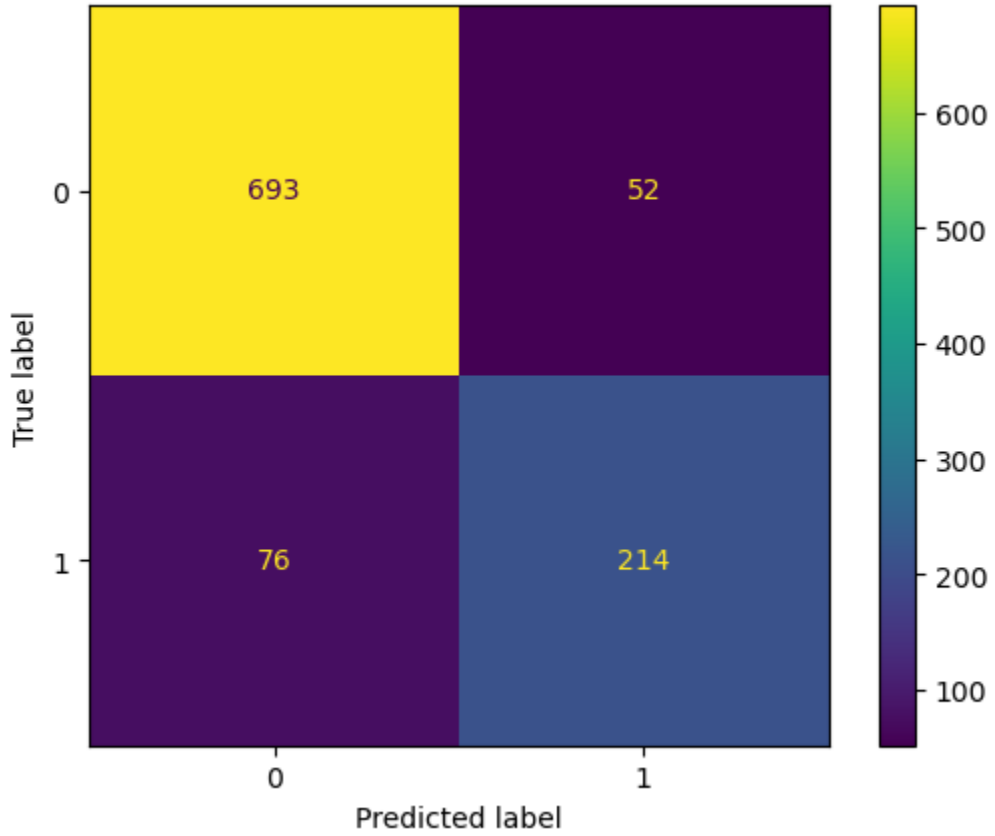
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Out[16]:
0.8763285024154589

In [17]:
```
print("ConfusionMatrix:")
cm = confusion_matrix(y_test, y_pred)
cm
```

ConfusionMatrix:
Out[17]:
```
array([[693,  52],
       [ 76, 214]])
```

In [18]:
```
mat = ConfusionMatrixDisplay(confusion_matrix=cm)
mat.plot()
plt.show()
```



In [19]:
```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.90      0.93      0.92       745
           1       0.80      0.74      0.77       290

    accuracy                           0.88      1035
   macro avg       0.85      0.83      0.84      1035
weighted avg       0.87      0.88      0.87      1035
```

In [20]:
```
print("accuracy_score:")
accuracy_score(y_test, y_pred)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

accuracy_score:

Out[20]:

0.8763285024154589

In [21]:

```
print("precision_score:")
precision_score(y_test, y_pred)
```

precision_score:

Out[21]:

0.8045112781954887

In [22]:

```
print("recall_score:")
recall_score(y_test, y_pred)
```

recall_score:

Out[22]:

0.7379310344827587

In [23]:

```
print("Error:")
1 - accuracy_score(y_test, y_pred)
```

Error:

Out[23]:

0.1236714975845411

In [29]:

```
from sklearn.svm import SVC

SVM = SVC(gamma='auto')
SVM.fit(X_train, y_train)
```

Out[29]:

SVC

▶ Parameters

In [30]:

```
SVM.score(X_train, y_train)
SVM.score(X_test, y_test)
```

Out[30]:

0.9091787439613527

In [36]:

```
print("ConfusionMatrix:")
cm = confusion_matrix(y_test, y_pred)
cm
```

ConfusionMatrix:

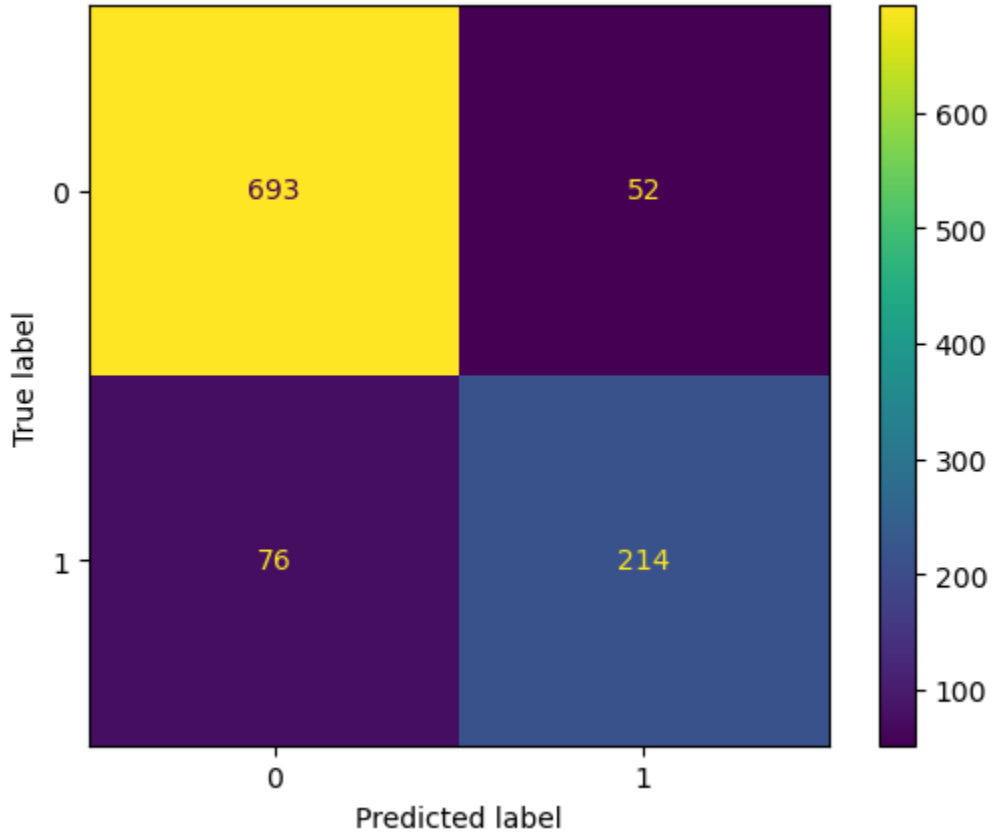
Out[36]:

```
array([[693,  52],
       [ 76, 214]])
```

In [37]:

```
mat = ConfusionMatrixDisplay(confusion_matrix=cm)
mat.plot()
plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



In [38]:

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.90      0.93      0.92       745
           1       0.80      0.74      0.77       290

    accuracy                           0.88      1035
   macro avg       0.85      0.83      0.84      1035
weighted avg       0.87      0.88      0.87      1035
```

In [ ]:

```

```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF